

HPC Tools for AI

Baseline BERT/SQuAD en una sola GPU

Juan-Manuel TOUYÁA TALAVERA

Plataforma: CESGA FinisTerae III
Modelo: `google-bert/bert-base-uncased`
Dataset: SQuAD v1.1
Framework: PyTorch + Hugging Face Transformers
Hardware: 1 × NVIDIA A100-PCIE-40GB

Resumen

Este informe establece una baseline reproducible de una sola GPU para el fine-tuning de BERT sobre SQuAD antes de pasar a entrenamiento distribuido. El estudio analiza el throughput de entrenamiento, la utilización de GPU, el uso de memoria, los modos de precisión, el tamaño de batch, la configuración del DataLoader, `torch.compile` y la estabilidad temporal utilizando exactamente una GPU NVIDIA A100. La baseline final utiliza BF16, un batch size por dispositivo de 160 y una época completa de SQuAD. En tres repeticiones válidas, el tiempo medio de entrenamiento fue de **234.58 s**, con un **coeficiente de variación del 0.42 %**. Un experimento independiente orientado a calidad alcanzó **79.56 Exact Match** y **87.50 F1** sobre validación de SQuAD.

1 Objetivos

El objetivo de esta baseline es caracterizar el rendimiento del fine-tuning de BERT sobre un único acelerador antes de introducir ejecución distribuida. El experimento fue diseñado para hacer fine-tuning de `google-bert/bert-base-uncased` sobre SQuAD v1.1, ejecutarse sobre exactamente una NVIDIA A100, medir el tiempo de pared de la fase de entrenamiento, identificar cuellos de botella relevantes y proporcionar una referencia reproducible para futuros experimentos de entrenamiento distribuido.

Una primera serie piloto fue descartada porque la configuración de SLURM hacía visibles las dos GPUs de un nodo con dos A100 al proceso de entrenamiento. Esas medidas se conservan únicamente con fines de diagnóstico y no se incluyen en los resultados válidos de una sola GPU.

2 Entorno experimental

2.1 Stack software

El entorno virtual dedicado se encuentra en `~/HPCTools/HPCTools4AI/.venv`.

Componente	Versión
Python	3.10.8
PyTorch	2.11.0+cu128
CUDA build	12.8
Transformers	5.17.0
Datasets	5.0.1
Accelerate	1.15.0
GPU	NVIDIA A100-PCIE-40GB

2.2 Preprocesamiento del dataset

El split completo de entrenamiento de SQuAD contiene 87,599 ejemplos originales y produce 88,492 features tokenizadas con el preprocesamiento mediante ventana deslizante adoptado. La longitud máxima de secuencia es 384 tokens, el document stride es 128 y se utiliza padding fijo a la longitud máxima.

3 Configuración final de la baseline

Parámetro	Valor
Modelo	<code>google-bert/bert-base-uncased</code>
Dataset	SQuAD v1.1
GPU	1 × NVIDIA A100-PCIE-40GB
Precisión	BF16
Épocas	1
Batch size por dispositivo	160
Learning rate	3×10^{-5}
DataLoader workers	0
Gradient accumulation	1
Optimizador	AdamW fused
Scheduler	Linear
<code>torch.compile</code>	Desactivado
TF32	Comportamiento por defecto de PyTorch

Una época completa ya dura claramente más de un minuto, por lo que una única época satisface el requisito de duración de la carga manteniendo razonables las repeticiones del benchmark.

4 Garantía de ejecución en una sola GPU en CESGA

La configuración oficial de SLURM solicita una A100 y 32 cores de CPU. El proceso Python se lanza mediante un paso `srun` explícito que solicita una A100. Algunos trabajos piloto utilizaban `-exclusive` a nivel de nodo, lo que hacía visibles ambas GPUs de un nodo con dos A100 al shell del batch y permitía al framework detectar dos aceleradores.

La implementación final utiliza dos salvaguardas: aislamiento de GPU en el paso `srun` y una comprobación en `train.py` que aborta si no hay exactamente un dispositivo CUDA visible.

5 Metodología de temporización

El benchmark principal mide únicamente el tiempo de pared de `trainer.train()`. Se ejecuta una sincronización CUDA inmediatamente antes y después de la región temporizada.

El tiempo reportado excluye carga del modelo, descarga/carga del dataset, tokenización, preprocesamiento y serialización final del modelo. De este modo se aísla la fase de fine-tuning y se mejora la comparabilidad entre repeticiones.

6 Estudio de optimización de rendimiento

6.1 Batch size

Con la configuración de precisión original, aumentar el batch size de 48 a 96 y 128 no mejoró de forma material el throughput y aumentó de manera considerable el uso de memoria GPU. Tras habilitar BF16, se seleccionó batch size 160 como punto práctico de equilibrio entre throughput y memoria: batch 192 no aportó prácticamente ninguna mejora adicional de velocidad y consumió más memoria.

6.2 DataLoader workers

Utilizar 0, 2 o 4 workers produjo tiempos prácticamente idénticos, alrededor de 59.2 s sobre el subconjunto controlado. El pipeline de entrada no constituía un cuello de botella relevante, por lo que se mantuvo `num_workers=0`.

6.3 BF16

Precisión	Tiempo medio	Memoria asignada máxima
Precisión por defecto	~59.64 s	~12.46 GiB
BF16	~12.26 s	~8.61 GiB

BF16 proporcionó aproximadamente un **speedup de $4.87\times$** y redujo la memoria GPU máxima asignada.

6.4 TF32

TF32 explícito redujo el tiempo medio de la ruta de precisión por defecto de aproximadamente 59.63 s a 19.55 s. BF16 siguió siendo más rápido y más eficiente en memoria, por lo que fue seleccionado para la baseline oficial.

6.5 torch.compile

Modo	Tiempo medio de entrenamiento
Eager	~87.30 s
Compilado	~151.31 s

Para la carga corta evaluada, el coste de compilación dominó y volvió más lenta la ruta compilada. Un experimento de tres épocas redujo la penalización relativa, pero no modificó el resultado global. También se observó una recompilación para el último batch irregular. Por ello, `torch.compile` se desactivó en la ejecución de referencia.

7 Resultados de profiling

Métrica	Valor representativo
Utilización GPU media	~89.6 %
Utilización GPU ≥ 80 %	~97.6 % de las muestras GPU
Memoria GPU máxima	~26.8 GiB
Potencia GPU media	~238 W

La utilización sostenida de GPU y la ausencia de mejoras medibles al aumentar los DataLoader workers apoyan la conclusión de que la fase temporizada de entrenamiento está principalmente limitada por GPU.

8 Resultados oficiales de la baseline

Repetición	Tiempo de entrenamiento
1	233.48 s
2	235.41 s
3	234.85 s

Estadística	Valor
Media	234.58 s
Mediana	234.85 s
Desviación estándar muestral	0.99 s
Coficiente de variación	0.42 %

La baseline final de una sola A100 es aproximadamente **234.6 segundos, es decir, 3.91 minutos, por época completa de SQuAD**. El bajo coeficiente de variación indica una buena estabilidad entre repeticiones.

9 Experimento de control de calidad

La baseline orientada a throughput no pretende maximizar la calidad del modelo. Una ejecución separada orientada a calidad utilizó el dataset completo de SQuAD, batch size 12, dos épocas, learning rate 3×10^{-5} , aproximadamente 10 % de warmup, BF16, AdamW fused, weight decay 0.01 y validación después de cada época.

La validation loss fue 0.9823 tras la época 1 y 0.9893 tras la época 2, por lo que se seleccionó el checkpoint de la época 1 como mejor modelo.

Este experimento muestra la diferencia entre optimización de throughput HPC y generalización de machine learning. Como varios hiperparámetros cambiaron simultáneamente, la mejora de calidad no debe atribuirse a un único factor de forma aislada.

Métrica	Resultado
Exact Match	79.56
F1	87.50

10 Reproducibilidad

Todas las ejecuciones válidas de rendimiento fuerzan exactamente una GPU CUDA visible. La semilla aleatoria utilizada es 42. Pueden permanecer pequeñas diferencias numéricas porque los kernels GPU y componentes internos del framework no están garantizados como deterministas bit a bit.

Los resultados generados, checkpoints, archivos TensorBoard, entornos virtuales y pesos de modelos se excluyen intencionadamente de Git. Las medidas experimentales detalladas se documentan en `EXPERIMENTS.md`.

11 Conclusión

La configuración final de referencia utiliza BERT-base-uncased sobre SQuAD v1.1 completo, una NVIDIA A100-PCIE-40GB, BF16, batch size 160 y una época. El tiempo medio de entrenamiento es aproximadamente **234.6 s**, con un **coeficiente de variación del 0.42 %**.

BF16 fue la optimización más importante, proporcionando aproximadamente un **speedup de $4.87\times$** sobre el subconjunto controlado. El paralelismo del DataLoader no produjo una mejora medible y `torch.compile` no resultó ventajoso para el benchmark corto porque el coste de compilación dominó.

La ejecución independiente orientada a calidad alcanzó **79.56 EM / 87.50 F1**. En conjunto, estos resultados proporcionan una referencia HPC reproducible de una sola GPU y una comprobación de calidad de machine learning para la futura fase de entrenamiento distribuido.